



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

GENEROVÁNÍ KÓDU Z TEXTOVÉHO POPISU FUNKCIONALITY

GENERATING CODE FROM TEXTUAL DESCRIPTION OF FUNCTIONALITY

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

JAKUB KOŘÍNEK

VEDOUCÍ PRÁCE

SUPERVISOR

doc. RNDr. PAVEL SMRŽ, Ph.D.

BRNO 2025

Zadání bakalářské práce



160532

Ústav: Ústav počítačové grafiky a multimédií (UPGM)
Student: **Kořínek Jakub**
Program: Informační technologie
Název: **Generování kódu z textového popisu funkcionality**
Kategorie: Informační systémy
Akademický rok: 2024/25

Zadání:

1. Seznamte se se způsoby využití rozsáhlých jazykových modelů typu GPT-3 a jejich adaptace na doménových datech, např. archivu GitHub a StackExchange.
2. Zpracujte dostupná data komentovaných kódů tak, aby bylo možné průběžně vyhodnocovat výsledky vytvářených modelů.
3. Na základě získaných poznatků navrhnete a implementujete systém, který dokáže na základě hlaviček funkcí a prvotních komentářů navrhovat strukturu následného kódu.
4. Vyhodnotíte výsledky systému na reprezentativním vzorku dat, diskutujete závislost výsledků na intuitivní složitosti popisu.
5. Vytvořte stručný plakát prezentující práci, její cíle a výsledky.

Literatura:

- dle doporučení vedoucího

Při obhajobě semestrální části projektu je požadováno:

- funkční prototyp řešení

Podrobné závazné pokyny pro vypracování práce viz <https://www.fit.vut.cz/study/theses/>

Vedoucí práce: **Smrž Pavel, doc. RNDr., Ph.D.**
Vedoucí ústavu: Černocký Jan, prof. Dr. Ing.
Datum zadání: 1.11.2024
Termín pro odevzdání: 14.5.2025
Datum schválení: 12.11.2024

Abstrakt

Tato práce se zabývá vývojem multiagentního systému pro generování kódu na základě vstupního dotazu uživatele. Systém umožňuje uživateli personalizovat výstup pomocí zpětné vazby. Pro generování i opravu kódu jsou využity velké jazykové modely založené na architektuře Transformer. Přestože výsledný multiagentní systém není přímým konkurentem moderních komerčních řešení, jako je GPT-4o, demonstruje přínos agentní spolupráce a možnosti využití vlastních modelů.

Abstract

This thesis focuses on the development of a multi-agent system for code generation based on user input. The system enables output personalization through user feedback. Large language models based on the Transformer architecture are used for both code generation and correction. While the resulting multi-agent system is not a direct competitor to modern commercial solutions such as GPT-4o, it demonstrates the benefits of agent collaboration and the potential of leveraging custom models.

Klíčová slova

generování kódu, oprava kódu, Transformer, LLM, CodeBERT, agentní systémy, NLP, strojové učení, umělá inteligence

Keywords

code generation, code correction, Transformer, LLM, CodeBERT, agent systems, NLP, machine learning, artificial intelligence

Citace

KOŘÍNEK, Jakub. *Generování kódu z textového popisu funkcionality*. Brno, 2025. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce doc. RNDr. Pavel Smrž, Ph.D.

Generování kódu z textového popisu funkcionality

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně pod vedením doc. Pavla Smrže. Uvedl jsem všechny literární prameny, publikace a další zdroje, ze kterých jsem čerpal.

.....

Jakub Kořínek

14. května 2025

Poděkování

Chci poděkovat vedoucímu práce doc. Pavlu Smržovi za nabídku tématu a podrobné rozebrání problematiky.

Obsah

1	Úvod	4
2	Neuronová síť	5
2.1	Výpočet výstupu neuronu	7
2.2	Učení neuronové sítě	9
3	Zpracování přirozeného jazyka	11
3.1	Tokenizace	11
3.2	Vektorová reprezentace slov	13
3.3	Minimální editační vzdálenost	13
4	Transformery a velké jazykové modely	15
4.1	Rekurentní neuronové sítě	15
4.2	Mechanismus pozornosti	16
4.3	Model Transformeru	17
5	Technologie a nástroje	21
5.1	Agentní a multiagentní systém	21
5.2	Quantized Low-Rank Adaptation	23
5.3	Použité datové sady	24
5.4	Evaluační metody	25
6	Návrh	27
6.1	Architektura systému	27
7	Implementační detaily	31
7.1	Trénink a vyhodnocování	31
7.2	Aplikace	33
8	Vyhodnocení výsledků	35
8.1	Vyhodnocení generování kódu	35
8.2	Vyhodnocení systému	35
8.3	Vyhodnocení personalizace	35
9	Závěr	37
	Literatura	38
A	Obsah paměťového média	41

Seznam obrázků

2.1	x je vstupem první vrstvy f_1 . Výstup této vrstvy je vstupem druhé vrstvy f_2 , atd. Posledním krokem je vrstva f_n , jejímž výstupem je výstup celého modelu.	5
2.2	Vizualizace neuronové sítě se skrytými vrstvami	6
2.3	Vizualizace výpočtu aktivace neuronu	7
2.4	Vizualizace aktivačních funkcí hyperbolický tangens a sigmoidální funkce	8
2.5	Vizualizace ReLU	9
2.6	Vizualizace dosměrné i zpětné propagace vedle sebe	10
3.1	Příklad využití tokenizace regulárních výrazů v NLTK Python-based toolkitu pro zpracování jazyka	12
3.2	Vizualizace Word2Vec architektury	13
3.3	Ukázka algoritmu pro nalezení minimální editační vzdálenosti	14
4.1	Ukázka zjednodušeného modelu Transformeru při překladu	16
4.2	Vizualizace Scaled Dot-Product Attention a Multi-Head Attention	17
4.3	Model transformeru	18
4.4	Ukázka metody Dropout pro potlačení přeučení	20
5.1	Jednoduchá vizualizace agentního systému	22
5.2	Vizualizace reparametrizace podle metody LoRa	24
6.1	Blokový diagram aplikace CodeAgent. Červeně jsou označeny součástky multiagentního systému. Jednotlivé komponenty jsou blíže popsány v dalších kapitolách.	28
6.2	Blokový diagram logiky generátoru a paměťového agenta s využitím modelu CodeGen.	29
6.3	Ukázka záznamu paměťového agenta	29
7.1	Graf vztahu ztráty k postupu epochami podle parametrů v sekci 7.1.2. Model byl laděn na jednom GPU GeForce RTX 4080. Proces trval přibližně 8 h a 34 m. Graf znázorňuje pouze první polovinu první epochy.	32
7.2	Ukázka aplikace CodeAgent	33
7.3	Ukázka generování kódu	34
8.1	Ukázka generování algoritmu výpočtu faktoriálu – Vlevo je vygenerovaný kód, který byl odmítnut, a vpravo nový kód. Byl vygenerován až na šestý pokus.	36

8.2	Ukázka generování algoritmu nalezení prvočísla – Obrázek vlevo ukazuje první vygenerování. Po odmítnutí zjevně chybného kódu byl na třetí přegenerování vygenerován kód na obrázku vpravo.	36
-----	--	----

Kapitola 1

Úvod

V posledních letech došlo k výraznému rozvoji v oblasti umělé inteligence, zejména v souvislosti s modely založenými na architektuře Transformer. Tyto modely nejenom napomáhají porozumění přirozenému jazyku, ale také jeho efektivnímu generování v různých kontextech. Rozvoj automatizované generace zdrojového na základě textového zadání či popisu má potenciál zefektivnit práci vývojářů, usnadnit výuku programování či umožnit rychlé prototypování softwarových řešení.

Cílem této práce je navrhnout, vyvinout a vyhodnotit multiagentní systém *CodeAgent* pro personalizované generování zdrojového kódu na základě zadání či popisu v přirozeném jazyce. Navržený systém kombinuje více modelových komponent: generátor kódu, korekčního agenta pro opravu chyb a paměťového agenta pro sledování předchozích výstupů. Klade přitom důraz na spolupráci mezi těmito komponentami s cílem zlepšit kvalitu výstupů.

Práce je strukturována následujícím způsobem: V následujících třech kapitolách je proveden výzkum teorie související s generováním kódu. Podrobně jsou rozebrány neuronové sítě a jejich součásti, zpracování přirozeného jazyka a tomuto oboru náležící algoritmy a neposlední řadě model transformeru a související velké jazykové modely. V projektu jsou představeny využití technologie a datové sady využití pro tvorbu modelů.

Následně práce prezentuje využití zmíněných technologií s podrobným návrhem systému. Ten je doplněn detaily o implementaci aplikace. Celá práce je zakončena evaluací výkonu systému.

Kapitola 2

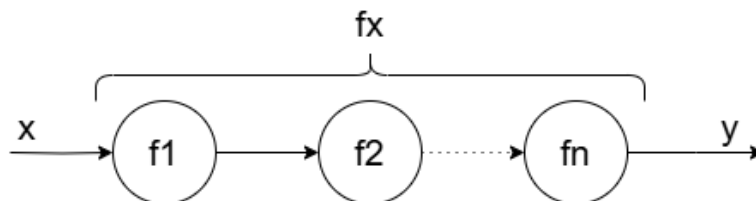
Neuronová síť

Neuronová síť, také známá jako umělá neuronová síť, je podmnožinou strojového učení a zároveň základem algoritmů hlubokého učení. V této kapitole se budu věnovat její struktuře, fungování a aplikaci, přičemž se také soustředím na dopředně orientované sítě, lépe známé jako *Feedforward networks*, dále jen FFN¹. Pokud není řečeno jinak, základní informace a obecný přehled budou vycházet z [11] a [16].

Síť se nazývá neuronová, protože její původ leží ve zjednodušeném modelu neuronu z práce McCullocha a Pittse [22] z roku 1943. Práce popisuje biologický neuron jako výpočetní jednotka, která lze definovat pomocí výrokové logiky. Pro každou takovou výpočetní jednotku pak existuje prahová hodnota (*threshold*), kterou musí překročit vstupní signál (*excitace*), aby neuron inicioval impuls. Moderní využití neuronových sítí však z těchto poznatků z oboru biologie čerpá jen velmi málo.

Cílem FFN jako modelu neuronové sítě je aproximovat funkci $y = f^*(x)$ tak, že definuje mapování $y = f(x; \theta)$ a zjistí hodnotu parametru θ , aby se výsledek funkce f byl co nejbližší výsledku funkce f^* .

Neuronové sítě jsou typicky reprezentovány složením mnoha různých funkcí. Model je spojen se směrovaným acyklickým grafem, který popisuje, jak se funkce skládají dohromady. Například můžeme mít tři funkce $f(1)$, $f(2)$ a $f(3)$, které jsou propojeny do řetězce, čímž vznikne $f(x) = f(3)(f(2)(f(1)(x)))$. Tyto řetězové struktury jsou nejběžněji používanými strukturami neuronových sítí. V tomto případě se $f(1)$ nazývá první vrstva sítě, $f(2)$ je druhá vrstva a tak dále. Celková délka řetězce určuje *hloubku modelu*². Název „hluboké učení“ vznikl právě z této terminologie. Poslední vrstva dopředné sítě se nazývá výstupní vrstva. Více v následujícím obrázku 2.1:



Obrázek 2.1: x je vstupem první vrstvy f_1 . Výstup této vrstvy je vstupem druhé vrstvy f_2 , atd. Posledním krokem je vrstva f_n , jejímž výstupem je výstup celého modelu.

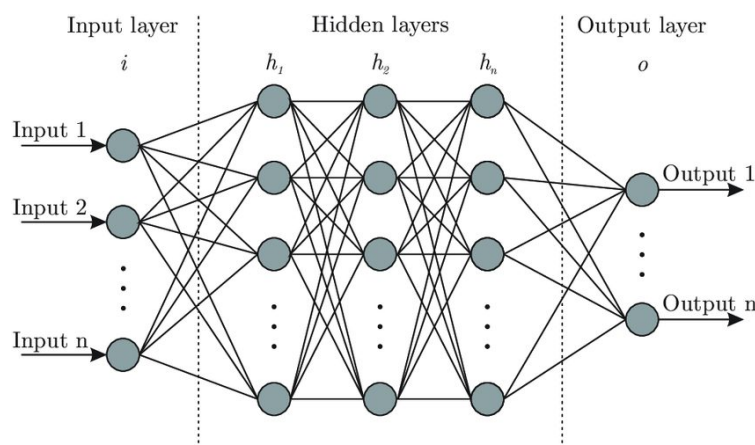
¹Jedná se o necyklickou variantu neuronových sítí. To znamená, že neobsahují propojení s předcházejícími vrstvami sítě.

²Vstupní vrstva se do hloubky běžně nepočítá.

Vstupy a výstupy jednotlivých vrstev jsou ale běžně vyhodnocovány jako vektory. Každá jednotka v těchto vektorech může být chápána jako uzel, který operuje paralelně s ostatními uzly ve stejné vrstvě. Právě tyto uzly jsou nazývány *neurony*.

V průběhu trénování neuronové sítě se snažíme dosáhnout toho, že se $f^*(x)$ bude rovnat $f(x)$. Trénovací data (*dataset*) nám poskytují vyhodnocené příklady $f^*(x)$ v různých fázích trénování, přičemž každý příklad x je doplněn hodnotou $y \approx f^*(x)$. Jednotlivé záznamy trénovacích dat přesně specifikují, jak se má chovat výstupní vrstva pro všechna x ; jejím výstupem musí být hodnota, která se blíží y .

Chování ostatních vrstev není přímo specifikováno trénovacími daty. Učicí algoritmus musí vyhodnotit, jak využít tyto vrstvy k produkci požadovaného výstupu, ale trénovací data neuvádějí, co by každá jednotlivá vrstva měla dělat. Místo toho musí učicí algoritmus vyhodnotit, jak využít tyto vrstvy k co nejlepší implementaci aproximace f^* . Protože trénovací data neukazují požadovaný výstup pro každou z těchto vrstev, nazývají se *skryté vrstvy*.



Obrázek 2.2: Vizualizace neuronové sítě se skrytými vrstvami – datová sada určuje data na vstupní vrstvě (zde Input layer) a očekávané chování vrstvy výstupní (zde Output layer), ale neurčuje výstupy skrytých vrstev (zde Hidden layers). Mezi neurony ve stejné vrstvě pak není žádné spojení.³

Cílem skrytých vrstev je zajistit schopnost sítě rozmělnit i potenciálně velmi složité problémy na lépe zvládnutelné podproblémy, které si mohou jednotlivé vrstvy rozdělit.

Pokud se však omezíme pouze na lineární operace, pak je model schopen porozumět pouze lineárním závislostem. Jinými slovy, model není schopen porozumět interakci jakýchkoliv dvou vstupních proměnných.

Aby model mohl reprezentovat nelineární funkci proměnné x , můžeme aplikovat lineární model ne na proměnnou x samotnou, ale na transformovaný vstup $\theta(x)$, kde θ je nelineární transformace.

Tato nelineární funkce se nazývá *aktivační funkce* 2.1.1. Konečný výstup neuronu se pak nazývá *aktivace*.

³Převzato z <https://viso.ai/deep-learning/neuron-activation/>

2.1 Výpočet výstupu neuronu

Základním blokem neuronových sítí je lineární vrstva. Ta vypočítává lineární transformaci jako:

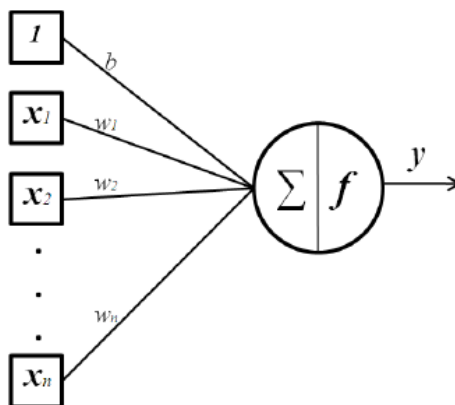
$$y = x^T \cdot w + b \quad (2.1)$$

, kde w a b dohromady reprezentují θ . w je často nazývána *váha* (*weight*) a slouží k posílení či potlačení aktivace přicházející z konkrétního spojení. b je označována jako *bias*. Bias ovlivňuje tendenci neuronu k větší či menší aktivaci. x^T je transponovaný vektor aktivací neuronů z předešlé vrstvy. A nakonec y je výstupní vektor aktivací neuronů.

Další důležitou vrstvou je vrstva nelineární. Je velmi podobná lineární transformaci, ale její výstup je ještě přepočítán nelineární funkcí. Aktivaci neuronu lze pak definovat následujícím způsobem:

$$\text{Aktivace} = \phi\left(\sum_{i=1}^n W_i x_i + b\right) \quad (2.2)$$

, kde ϕ je již zmíněná aktivační funkce, n je počet spojení z předešlé vrstvy k tomuto neuronu z neuronu i , W_i je váha tohoto spojení a x_i aktivace spojeného neuronu. b je pak bias neuronu, jehož aktivaci počítáme. Tato definice je vizualizována na následujícím obrázku 2.3.



Obrázek 2.3: Vizualizace výpočtu aktivace neuronu – Vstupem neuronu je suma n aktivací neuronů z předešlé vrstvy a váh jejich spojení a biasu tohoto neuronu. Výstup je následně přepočítán aktivační funkcí.⁴

2.1.1 Aktivační funkce

Tato sekce bude přejata z [9]. Aktivační funkce nemusí být příliš složitá. Porovnání oblíbených funkcí je vizualizováno v obrázcích 2.5 a 2.4. Mezi oblíbené aktivační funkce patří například:

⁴Převzato z https://www.researchgate.net/figure/General-model-of-the-artificial-neuron_fig1_263441297

- **Rectified linear unit (ReLU)** – Jedná se o nejjednodušší a nejčastěji využívanou aktivační funkci. Lze ji definovat jako:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{pro } x \geq 0 \\ 0 & \text{ostatní} \end{cases} \quad (2.3)$$

Obor hodnot ReLU je $[0, \infty)$. Algoritmus řeší problém výpočetní náročnosti jiných algoritmů, ale potýká se s tzv. *Dying ReLu Problem*. Ten nastává, když i výrazně záporné vstupy produkují nulový výstup, proto se tyto neurony nikdy neaktivují a tuto chybu už nejde přeučit.

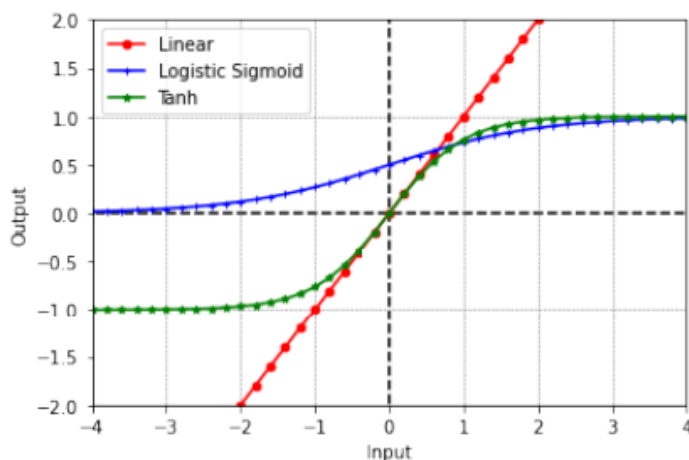
- **Sigmoidální funkce** – Tato funkce byla využívána hlavně v minulosti. Je definována jako:

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (2.4)$$

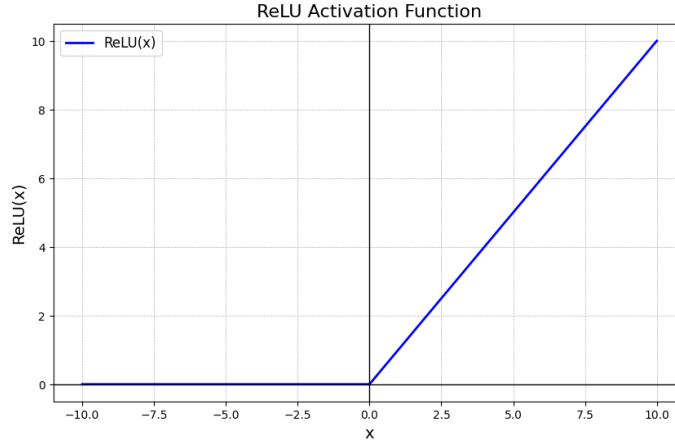
Obor hodnot je $[0, 1]$

- **Hyberbolický tangens** - Je podobná sigmoidální funkci, ale posouvá obor hodnot do $[-1, 1]$. Je definována jako:

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.5)$$



Obrázek 2.4: Vizualizace aktivačních funkcí hyperbolický tangens a sigmoidální funkce



Obrázek 2.5: Vizualizace ReLU

2.2 Učení neuronové sítě

Z definice lineární transformace 2.1 vyvstává otázka, jak zjistit optimální hodnotu θ . Na tento problém existovalo i existuje několik řešení.

- Jednou z možností je použít velmi obecnou funkci ϕ , například nekonečně dimenzionální ϕ , která je implicitně použita v kernelových metodách založených na RBF jádru. Pokud má $\phi(x)$ dostatečně vysokou dimenzi, můžeme mít vždy dostatečnou kapacitu pro naučení trénovací množiny, ale generalizace na testovací množinu často zůstává slabá. Velmi obecné mapování příznaků je obvykle založeno pouze na principu lokální hladkosti a nezahrnuje dostatek předběžných informací k řešení složitějších úloh.
- Další možností je ϕ ručně navrhnout. Až do nástupu hlubokého učení byla tato metoda převládající. Vyžaduje však příliš velké množství lidského úsilí a času pro každou samostatnou úlohu. Jakmile je úloha hotová, jakýkoliv posun se dá stěží sdílet mezi jednotlivými oblastmi.

Proces trénování neuronové sítě spočívá v iterativním přizpůsobování parametrů modelu, konkrétně vah a biasů jednotlivých neuronů, tak, aby výstup sítě co nejlépe odpovídal požadovaným hodnotám. Tento proces začíná tzv. *forward propagací*, při které vstupní data procházejí vrstvami sítě. Každý neuron počítá součet vah svých vstupů, přičte k němu bias a aplikuje nelineární aktivační funkci, například ReLU nebo sigmoidální funkci.

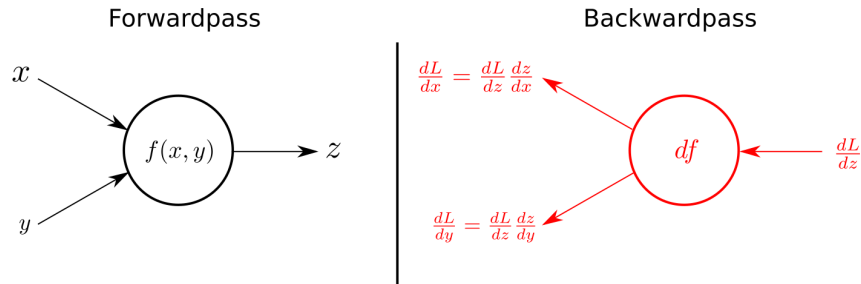
Po výpočtu výstupu následuje výpočet ztráty pomocí tzv. *ztrátové funkce* (*loss function*), která měří rozdíl mezi skutečným výstupem a predikcí modelu. V závislosti na typu úlohy se používají různé ztrátové funkce; jedna z nich je křížová entropie (*cross-entropy*) pro klasifikaci. Ta je definována:

$$\text{Cross-Entropy} = - \sum_{i=0}^n y_i \log(y_i^*) \quad (2.6)$$

, kde y_i je požadovaný výstup a y_i^* je předpovězený výstup.

Následuje klíčová fáze zvaná *zpětná propagace* (*backpropagation*). Pomocí pravidla řetězového derivování se spočítají gradienty ztráty vzhledem ke všem vahám v síti. Tyto

gradienty pak slouží k aktualizaci parametrů modelu směrem, který minimalizuje chybu. Tento krok se nazývá *gradientní sestup* (*gradient descent*).^[28]



Obrázek 2.6: Vizualizace dosměrné i zpětné propagace vedle sebe – pomocí funkce $f(x, y)$ spočítáme výstup z . Po obdržení $\frac{\partial L}{\partial z}$ (derivace celkové ztráty podle výstupu z) můžeme pomocí pravidla řetězového derivování (chain rule) spočítat jednotlivé gradienty vstupních proměnných x a y vzhledem ke ztrátové funkci.⁵

Celý tento proces se opakuje po dobu několika *epoch*, kdy každá epocha představuje jeden průchod celou trénovací množinou. Pro zrychlení a stabilitu se často používají *mini-batche*, které rozdělují data do menších skupin.

Jedním z problémů, kterému musíme při trénování čelit, je tzv. *overfitting*. Overfitting (přeučení) nastává tehdy, když je model vystaven nejen trénovacím datům, ale i šumu (nepodstatným nebo irrelevantním informacím obsaženým v těchto datech) do takové míry, že to negativně ovlivní jeho výkon na nových, neviděných datech. Šum představuje právě tyto irrelevantní údaje. To znamená, že model se učí jak relevantní vzory, tak i náhodný šum, což vede k přeučení a tím ke snížení přesnosti modelu při praktickém nasazení.^[1]

Opačný problém je situace, kdy se model nedokáže dostatečně dobře naučit ani trénovací data, ani zobecňovat nová data, která mu jsou předložena. Jelikož se kvalitně nenaučí ani trénovací sadu, bude vykazovat špatný výkon i na jakýchkoliv dalších datech. Tomuto jevu říkáme *underfitting* (nedostatečné učení).^[1]

⁵Obrázek převzat z článku <https://kratzert.github.io/2016/02/12/understanding-the-gradient-flow-through-the-batch-normalization-layer.html>

Kapitola 3

Zpracování přirozeného jazyka

Zpracování přirozeného jazyka (*Natural Language Processing*, NLP) je klíčový podobor umělé inteligence, který umožňuje strojům interpretovat lidský jazyk, rozumět mu a generovat jej. V kontextu moderních systémů umělé inteligence, zejména těch založených na velkých jazykových modelech (*Large Language Model*, LLM), hraje NLP zásadní roli při propojení surového textu a strukturovaných strojových reprezentací. Tradičně se NLP spoléhalo na systémy založené na pravidlech a statistických metodách pro zpracování jazyka. S příchodem hlubokého učení a architektur transformátorů však došlo k vývoji do oblasti, která je schopná podporovat složité aplikace, jako je strojový překlad, konverzační agenty a generování kódu.

Tato kapitola se zaměřuje na základy moderního NLP s důrazem na technologie, které jsou pro generativní systémy nejrelevantnější. Jsou zde představeny klíčové koncepty, jako je tokenizace, vektorová reprezentace slov a minimální editační vzdálenost. Kapitola zdůrazňuje přechod od ručního vytváření rysů k učení na základě celkového modelu, což činí NLP klíčovou součástí návrhu inteligentních adaptivních AI agentů. Pokud nebude řešeno jinak, kapitola vychází z [16].

3.1 Tokenizace

Zpracování textu znamená převedení textu do modelu srozumitelné standardizované formy. Většina operací, které s jazykem provádíme, začíná u rozdělení na jednotlivá slova, tedy u *tokenizace* slov nebo jejich částí. V českém nebo anglickém jazyce velmi často stačí oddělení pouze na základě bílých znaků. Například u japonštiny ale mezery mezi slovy vůbec nejsou.

Textová část je tedy převedena do tzv. *tokenů*. Modely, které zpracovávají text, pak zpravidla obsahují vrstvu pro vektorovou reprezentaci (*embedding layer*), která mapuje každý token v řadě do vektoru.

3.1.1 Přístup shora dolů

Tokenizace shora dolů (také známá jako *top-down* nebo *rule-based* tokenizace) je založena na rozdělení textu na větší, předdefinované jednotky. Typicky jsou tyto jednotky slova nebo věty. Tento proces pak využívá daná jazyková pravidla a předdefinované slovníky.

Tento způsob naráží na řadu problémů. Tečky jsou pro oddělení vět užitečné znaky, ale existují případy, kdy je potřebujeme zachovat jako součást tokenů¹. Speciální znaky a čísla

¹Příkladem může být titul Ph.D., m.p.h. ap.

musí být zachována v cenách a datech². Dalším příkladem jsou odkazy, e-mailové adresy a jiné.

Běžně používaný standard tokenizace je znám jako *Penn Treebank Tokenization Standard*[21], který poskytuje robustní zdroj datových sad. Řeší běžné problémy anglického jazyka jako například apostrof, taková slova rozděljuje do dvou tokenů³.

V praxi probíhá tokenizace před jakýmkoliv dalším zpracováním jazyka, proto musí být velmi rychlá. Obecně se používají deterministické algoritmy založené na regulárních výrazech kompilovaných do efektivního konečného automatu. Na příklad se můžeme podívat zde 3.1.

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     (?:[A-Z]\.)+        # abbreviations, e.g. U.S.A.
...     | \w+(?:-\w+)*      # words with optional internal hyphens
...     | \$?\d+(?:\.\d+)?%? # currency, percentages, e.g. $12.40, 82%
...     | \.\.\.            # ellipsis
...     | [][.,;'"'?()_:-' ] # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Obrázek 3.1: Příklad využití tokenizace regulárních výrazů v NLTK Python-based toolkitu pro zpracování jazyka.⁴

3.1.2 Přístup zdola nahoru

Tento způsob tokenizace textu je ve velkých jazykových modelech jeden z nejvíce využívaných. Místo aby definoval tokeny jako slova či znaky, používá svá data, aby nám automaticky řekl, jaký by token měl být. Tento přístup se vyplatí v situacích, kdy slovo neznáme.

Algoritmy využívající tento přístup často získávají informace o jazyce z jednoho *korpusu* (trénovacího korpusu) a následně tyto informace využívají, když se rozhodují v rámci separátního testovacího korpusu. Pokud trénovací korpus obsahuje slovo, které moderní tokenizátor nezná, automaticky vytváří sady tokenů, které zahrnují i tokeny menší než slova, tzv. *podřetivce*.

Mezi nejčastěji používané algoritmy patří například *byte-pair encoding*[30] nebo *unigram language modeling*[18], případně *SentencePiece*[19], který kombinuje předešlé dva.

²\$45.55, 1.1.2001, ap.

³doesn't se rozdělí na does + n't

⁴Převzato z [3].

3.2 Vektorová reprezentace slov

Na poli zpracování přirozeného jazyka byla vektorová reprezentace slov dlouho výzva. Hraje však důležitou roli zejména jako vstupní reprezentace pro neuronové sítě.

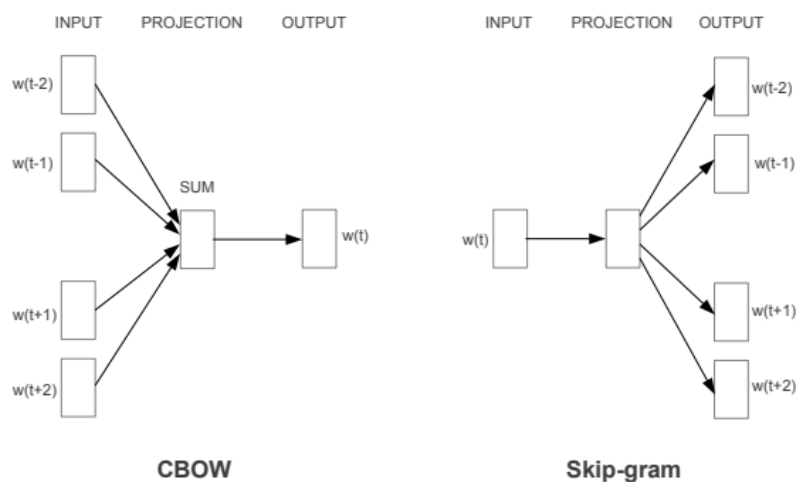
Jeden z nejznámějších a nejpoužívanějších modelů je *Word2Vec*. Metody word2vec jsou rychlé, efektivní na trénink, jednoduše dostupné online i s kódem a předtrénovanými reprezentacemi. Navíc se mu poprvé podařilo správně zachytit sémantické části slov, které reprezentují celý význam slova jako takového.[23]

Překvapení bylo, že podobnost slovních reprezentací přesahuje pouhou syntaktickou podobnost. Pomocí techniky posunu slov (*word offset*), které provádějí jednoduché algebraické operace s vektorovými reprezentacemi slov, se například ukázalo, že vektor(„Král“) - vektor(„Muž“) + vektor(„Žena“) vede k vektoru, který je nejbližší vektorové reprezentaci slova „Královna“. [24].

Word2vec přišel s dvěma architekturami modelu:

- **Continuous Bag-of-Word (CBOW)** – tato architektura předpovídá současné slovo na základě kontextu
- **Skip-gram** – tato architektura předpovídá okolní slova na základě daného slova

Architektury lze znázornit v následující vizualizaci:



Obrázek 3.2: Vizualizace Word2Vec architektur – V případě CBOW architektury vstupní vrstva produkuje výstup do projekční vrstvy, která je společná pro všechna slova. Jejím výstupem je výstup modelu. Skip-gram naopak předpovídá kontext na základě slova.⁵

3.3 Minimální editační vzdálenost

Kapitola se taktéž bude zabývat konceptem minimální editační vzdálenosti, jelikož je klíčový i pro tuto práci. Při zpracování jazyka se často zabýváme měřením vzdálenosti mezi dvěma řetězci.

⁵Převzato z [23]

Editační vzdálenost poskytuje způsob, jak kvantifikovat intuice ohledně podobnosti řetězců. Formálně je *minimální editační vzdálenost* mezi dvěma řetězci definována jako nejmenší počet editačních operací (například vložení, odstranění nebo nahrazení), které jsou potřeba k přeměně jednoho řetězce ke druhému.

Algoritmus pro nalezení nejmenší editační vzdálenosti můžeme chápat jako vyhledávací problém, kdy hledáme nejkratší cestu (sekvenci úprav) z jednoho řetězce ke druhému. Potenciální prostor pro všechny úpravy je enormní, proto nelze vyhledávat naivně. První algoritmus pro nalezení editační vzdálenosti byl pojmenován v [33]. Ukázku můžeme vidět na následujícím obrázku.

i n t e n t i o n	← delete i
n t e n t i o n	← substitute n by e
e t e n t i o n	← substitute t by x
e x e n t i o n	← insert u
e x e n u t i o n	← substitute n by c
e x e c u t i o n	

Obrázek 3.3: Cesta ze slova intention ke slovu execution. ⁶

⁶Převzato z [16]

Kapitola 4

Transformery a velké jazykové modely

Rychlý vývoj umělé inteligence je výrazně ovlivněn nástupem architektur založených na tzv. *Transformerech*, které dnes tvoří základ velkých jazykových modelů (LLM). Transformery byly poprvé představeny v roce 2017 v průlomové práci *Attention is All You Need*[32] a znamenaly revoluci ve zpracování přirozeného jazyka díky schopnosti modelovat dlouhodobé závislosti s vysokou výpočetní efektivitou.

Na základě této architektury vznikly modely jako GPT[5], BERT[8] nebo T5[27], které prokázaly výjimečné schopnosti v úlohách, jako je překlad, sumarizace, odpovídání na otázky nebo generování kódu. Tato kapitola se věnuje teoretickým základům transformerů, jejich rozšíření ve formě LLM a principům, které umožňují těmto modelům efektivně zpracovávat, chápat a generovat přirozený jazyk. Pochopení těchto aspektů je klíčové pro vývoj agentních systémů, které jazykové modely využívají pro specializované úlohy, například personalizované generování kódu.

Informace v této kapitole týkající se Transformerů budou vycházet z již zmíněné práce [32], obecné informace pak budou vycházet z práce [16].

4.1 Rekurentní neuronové sítě

Rekurentní neuronové sítě (RNNs) dříve představovaly standardní přístup pro zpracování sekvenčních dat, jako je strojový překlad, rozpoznávání řeči nebo generování textu. RNNs zpracovávají vstupy postupně; každý prvek sekvence ovlivňuje výstup skrze skrytý stav, který si uchovává informaci o předchozích prvcích. Díky této zpětné vazbě dokáží modelovat závislosti v čase, avšak pouze omezeně.

Transformery oproti tomu od rekurence zcela opouští. Místo postupného zpracování využívají mechanismus pozornosti, který umožňuje každému tokenu přímo přistupovat k celé vstupní sekvenci zároveň. Tomuto mechanismu se budu věnovat v další sekci 4.2.

RNNs ale trpí problémy s dlouhodobými závislostmi kvůli jevu zvanému *exploding* nebo *vanishing gradients*. To vedlo k vývoji vylepšených architektur, jako jsou LSTM (*Long Short-Term Memory*)[13] a GRU (*Gated Recurrent Unit*),[6] které lépe zvládají delší kontext.

4.2 Mechanismus pozornosti

V předešlých kapitolách 3.2 jsem zmiňoval model Word2vec, který staticky konvertoval slova do jejich vektorové podoby. Vektorová reprezentace stejného slova byla tedy pořád stejný vektor, ať už je kontext jakýkoliv. V následující části zmíním přeložený a pro účely českého jazyka lehce upravený příklad z práce [16]:

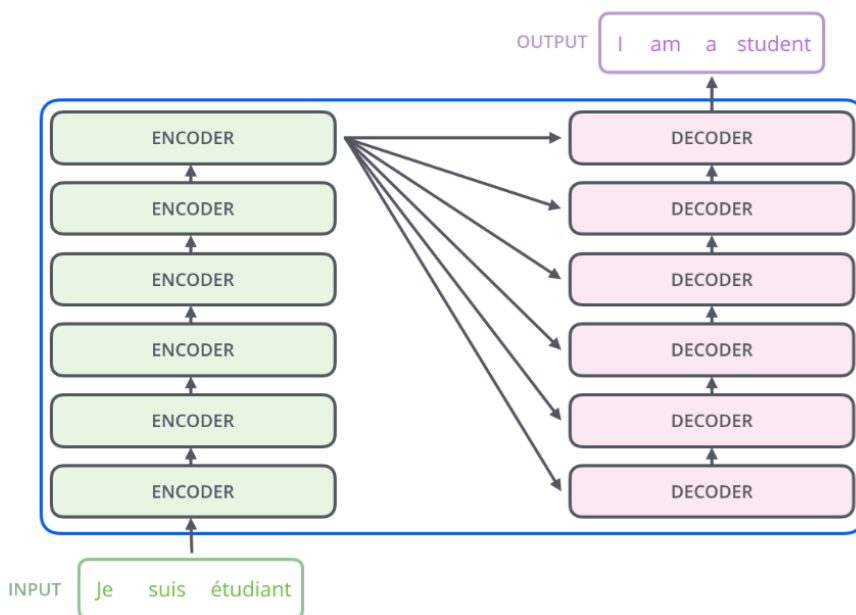
Slepice nepřekročila cestu, protože ona byla moc unavená. (4.1)

Slepice nepřekročila cestu, protože ona byla moc široká. (4.2)

Ve větě 4.1 je důraz věty na slově slepice, zatímco ve větě 4.2 je na slově cesta. Jinými slovy, pokud bychom se pokusili spočítat význam této věty, potřebujeme vědět, k čemu se vztahuje zájmeno „ona“.

Smyslem uvedeného příkladu je poukázat na to, že kontextová slova, která napomáhají určení významu daného slova, se mohou v rámci věty i odstavce nacházet i ve značné vzdálenosti od daného slova. Architektura Transformerů umožňuje vytvářet *kontextové reprezentace významu slov*, tzv. *kontextové embeddingy*, prostřednictvím integrace významu těchto podpůrných slov. V rámci transformeru dochází k postupnému budování stále bohatších a přesnějších kontextualizovaných reprezentací významu vstupních tokenů v jednotlivých vrstvách modelu.

Funkci pozornosti lze popsat jako zobrazení dotazu (*query*) a množiny dvojic klíč–hodnota (*key–value*) na výstup, přičemž dotaz, klíče, hodnoty i výstup jsou reprezentovány jako vektory. Výstup je vypočítán jako součet hodnot vah, kde váha je přiřazena jednotlivým hodnotám pomocí funkce kompatibility mezi dotazem a odpovídajícím klíčem.



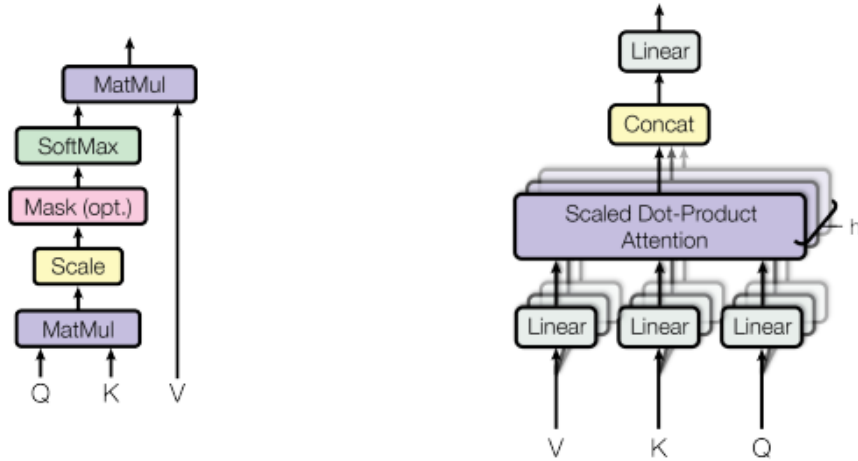
Obrázek 4.1: Ukázka zjednodušeného modelu Transformeru při překladi – Transformer obsahuje enkodér, který čte vstupní větu a generuje její reprezentaci. Finální stav enkodéru je zároveň první stav dekodéru. Dekodér následně generuje výstupní větu iterativně slovo po slově ve spolupráci s reprezentací vygenerovanou enkodérem. ¹

¹Obrázek převzat z <https://jalammar.github.io/illustrated-transformer/>

4.3 Model Transformeru

Objev mechanismu pozornosti byl v oboru LLM velký průlom. Dalším důležitým objevem bylo, že když byl mechanismus pozornosti doplněn o jednovrstvou dopřednou neuronovou síť, dosáhl model v překladových úlohách lepšího výkonu i bez použití složitějších architektonických prvků. Transformer navíc vykazoval vyšší výpočetní efektivitu, jelikož kvůli absenci *rekurentních* (zpětnovazebních) spojení umožňoval efektivní paralelizaci výpočtů.

Transformer používá svou vlastní variantu mechanismu pozornosti, která se nazývá *Multi-Head attention*, samotný vzorec pro výpočet se nazývá *scaled dot-product attention*. Lépe tento vztah znázorňuje následující obrázek. 4.2.



Obrázek 4.2: Scaled Dot-Product attention na levé straně a Multi-Head Attention na pravé. Ta se skládá z několika vrstev pozornosti běžících paralelně. ²

Vstupem mechanismu Scaled Dot-Product Attention jsou dotazy (*queries*) a klíče (*keys*) dimenze d_k , hodnoty dimenze jsou d_v . Vypočítáme skalární součin dotazů se všemi klíči, každý vydělíme $\sqrt{d_k}$. Následně aplikujeme funkci *softmax* pro výpočet vah přiřazených jednotlivým hodnotám.

V praxi se funkce pozornosti počítá pro množinu dotazů najednou, která je zabaleny do matice Q . Klíče a hodnoty jsou obdobně zabaleny do matic K a V . Výstupní matici poté získáme následovně:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK}{\sqrt{d_k}}\right)V \quad (4.3)$$

Dvě v praxi nejčastěji používané funkce pozornosti jsou *Additive Attention*[2] a *Dot-Product (Multiplicative) Attention*. Dot-Product Attention je identická s výše uvedeným algoritmem, s výjimkou škálovacího faktoru $\frac{1}{\sqrt{d_k}}$. Additive Attention počítá funkci kompatibility pomocí *feed-forward* sítě s jednou skrytou vrstvou.

Pro nízké hodnoty d_k mají oba mechanismy podobný výkon, ale podle [4] má additive attention lepší výkon pro vysoké hodnoty d_k .

Místo jediné funkce pozornosti nad klíči, hodnotami a dotazy o rozměru d_{model} , se ukázalo jako výhodné je lineárně projektovat h -krát do prostorů o rozměrech d_k a d_v .

²Převzato z [32]

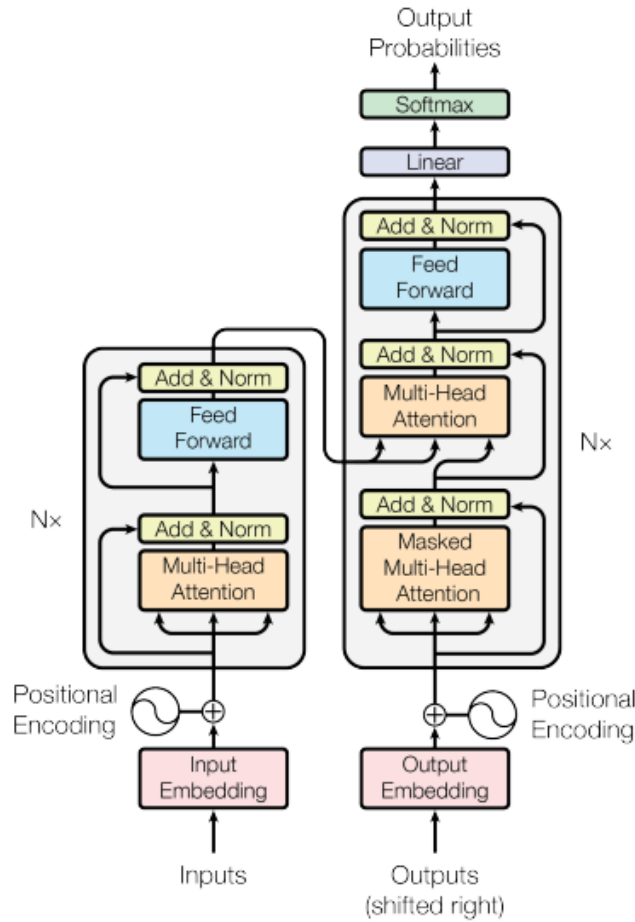
Funkce pozornosti se poté provádí paralelně na každé projekci, výstupy o rozměru d_v se zkonkatenují a znovu lineárně transformují, čímž vznikne finální výstup:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n)W^O \quad (4.4)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (4.5)$$

Základní článek[32] uvádí, že využívá $h = 8$ paralelních vrstev pozornosti. Pro každou z nich pak platí $d_k = d_v = d_{\text{model}}/8 = 64$. Díky zmenšenému rozměru každé hlavy je celková výpočetní náročnost srovnatelná s jednou hlavou s plnou dimenzionalitou.

Transformer využívá *encoder-decoder* strukturu 4.3.



Obrázek 4.3: Model transformeru – Enkodér je levá část modelu a dekodér práva část modelu. ³

4.3.1 Enkodér

Enkodér mapuje vstupní sekvenci reprezentace symbolů $(x_1, \dots, x_n)$ na sekvenci spojitých reprezentací $z = (z_1, \dots, z_n)$. Typicky se skládá z více identických vrstev. Základní článek uvádí vrstev šest. Každá vrstva má dvě podvrstvy.

³Převzato z [32]

První podvrstvou je Multi-Head Attention mechanismus a druhou je jednoduchá plně propojená feed-forward síť. Okolo každé podvrstvy je aplikováno reziduální spojení, následované normalizací vrstvy.

Všechny podvrstvy a embedding vrstvy v modelu mají výstupy o rozměru $d_{model} = 512$.

Jelikož náš model neobsahuje rekurenci ani konvoluci, je nutné do něj vložit informaci o pořadí tokenů ve vstupní sekvenci, aby mohl pracovat s jejich posloupností. Za tímto účelem je ke vstupním embeddingům přidáno tzv. poziční kódování (*positional encoding*) na spodní úrovni enkodéru i dekodéru. Poziční kódování má stejnou dimenzi jako embeddingy, takže jej lze s embeddingy jednoduše sečíst.

4.3.2 Dekodér

Dekodér generuje výstupní sekvenci $(y_1, \dots, y_n)$ symbolů iterativně na základě kontextu, který mu poskytnul enkodér. Jelikož model přijímá i předešlé vygenerované symboly jako dodatečný vstup při generaci textu, nazveme ho *auto-regresivní*.^[12]

Struktura bloku dekodéru se od bloku enkodéru liší jen málo. Zpravidla se skládá z více vrstev, podle základního článku taktéž ze šesti. Kromě dvou podvrstev dekodér přidává třetí vrstvu, která provádí Multi-Head Attention nad výstupem zásobníku enkodéru. Podobně jako u enkodéru používáme reziduální spojení okolo každé podvrstvy, následované normalizací vrstvy. Dále upravujeme self-attention podvrstvu v zásobníku dekodéru tak, aby jednotlivé pozice nemohly přistupovat k následujícím pozicím. Toto maskování, spolu se skutečností, že výstupní embeddingy jsou posunuty o jednu pozici, zajišťuje, že predikce pro pozici i může záviset pouze na známých výstupech na pozicích menších než i .

4.3.3 Trénování modelu

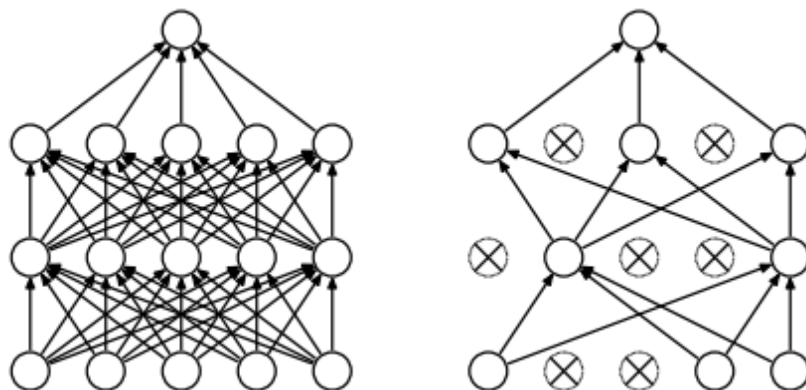
Trénink transformerů probíhá podobným způsobem jako u jakékoliv jiné neuronové sítě využívající gradientní sestup. Každý trénovací příklad je dvojice řetězců: zdrojový a cílový. Po spojení pomocí speciálního oddělovacího tokenu mohou tyto dvojice sloužit jako trénovací data.

Při *interferenci* (tedy během generování) používá dekodér jako vstup pro další časový krok x_{t+1} svou vlastní odhadovanou výstupní hodnotu $\hat{y}_t$. Tím se ale může postupně odchýlovat od správné cílové věty, protože se chyby mohou kumulovat. Proto se při trénování častěji používá tzv. *teacher forcing*. To znamená, že se místo předchozího výstupu dekodéru jako vstup x_{t+1} použije skutečný správný token z trénovací sady. Tento přístup urychluje učení, protože model se učí predikovat další token na základě správného předchozího kontextu, nikoliv vlastních (mnohdy chybných) výstupů.

Při trénování se často používá cross-entropy loss jako ztrátová funkce, což umožňuje modelu optimalizovat pravděpodobnosti správného výstupu.

Transformery používají optimizér Adam^[17], který sleduje jak průměrné hodnoty gradientů, tak jejich rozptyl, a adaptivně podle nich upravuje velikost kroků učení pro každý parametr zvlášť. Metoda je snadno implementovatelná, výpočetně efektivní, má nízké paměťové nároky, je invariantní vůči diagonálnímu přeskokování gradientů a je vhodná pro úlohy s velkým množstvím dat a/nebo parametrů.

Existuje hned několik metod, jak při tréninku předejít přeučení. Tento model využívá tzv. *Dropout* ^[31], jenž pracuje na principu náhodného „vypínání“ neuronů (včetně jejich spojení) v neuronové síti při trénování, což jim brání v přílišné koadaptaci.



Obrázek 4.4: Ukázka metody Dropout pro potlačení přeučení – Na levé straně je standardní neuronová síť s 2 skrytými vrstvami. Na pravé straně je příklad zředěné sítě po aplikaci metody Dropout na síť vlevo. Neurony s křížem byly „vypnuty“. ⁴

V architektuře Transformer je také běžnou praxí používat tzv. *batching*, což znamená zpracovávat více sekvencí najednou, čímž se zrychluje trénování a efektivně využívá výpočetní kapacita. Aby byly sekvence ve skupině (batchi) kompatibilní, musí mít všechny stejnou délku. Pro tento účel se používá tzv. *padding*, kde se kratší sekvence doplňují speciálním tokenem⁵, který nenese žádnou informaci. Během trénování je pak nutné použít maskování, které zajistí, že model nebude při výpočtu pozornosti brát v úvahu padding tokeny. Tato technika zajišťuje, že model nebere v úvahu nepodstatné části vstupu a dokáže se soustředit na skutečné informace ve vstupních sekvencích.

⁴Převzato z [31].

⁵Například <PAD>.

Kapitola 5

Technologie a nástroje

Pro implementaci neuronových sítí byly zvažovány dvě nejrozšířenější knihovny – PyTorch¹ a TensorFlow². Obě poskytují flexibilní rozhraní pro konstrukci modelů a podporují výpočty na GPU. Vzhledem k větší komunitní podpoře a jednodušší práci s dynamickými výpočetními grafy byl v projektu upřednostněn PyTorch. Ten umožnil snadnou integraci jak s vlastním trénovacím cyklem, tak s knihovnami jako `transformers` od Hugging Face³.

Tato knihovna poskytuje širokou škálu předtrénovaných jazykových modelů založených na architektuře Transformer včetně modelů jako BERT[8], GPT[5] nebo CodeBERT[10]. V projektu byla využita pro práci s předtrénovaným generativním modelem, který slouží jako základní komponenta pro generování kódu i pro následnou rozhodovací agentní síť po vygenerování. Kromě toho knihovna obsahuje nástroje pro jemné doladění (*fine-tuning*), správu tokenizace a jednoduché API pro nasazení modelů.

Další knihovny jazyka Python, které byly v tomto projektu využity, jsou `transformers`, `numpy`, `more_itertools`, `datasets`, `accelerate`, `peft`, `bitsandbytes`, `openai`, `toml`, `SentencePiece`, `difflib`.

Než se v této kapitole začnu věnovat optimalizačním algoritmům, ladění, využitým datovým sadám, ap., je nutné představit poměrně nový koncept agentních a multiagentních systémů.

5.1 Agentní a multiagentní systém

Pokud nebude uvedeno jinak, informace v této sekci budou vycházet z [29]. *Agentní systém* je dán prostředím, kde působí agent vybavený určitou dávkou inteligence, jež je využita k řešení problému. *Agent* je pak definován jako aktivní prvek systému vytvořený člověkem za určitým účelem. Výsledkem činnosti agenta je změna agentního systému z aktuálního do cílového stavu. Agent vnímá stav prostředí skrze *senzory* a jedná prostřednictvím *aktuátorů*, jak je vidět na obrázku 5.1. Agentu měříme jeho konkrétním měřítkem výkonu (*performance measure*). Všechny výše uvedené součástky agenta se sjednocují pod hlavičku tzv. *úlohového prostředí* (*task environment*). Obecně se označuje akronymem PEAS (performance, environment, actuators, sensors).

Koncept agentů však dalece přesahuje svět umělé inteligence. [29] uvádí příklad řidiče taxi. Typ agenta je v tomto případě řidič taxi. Jeho měřítko výkonu je bezpečnost, rychlost,

¹<https://pytorch.org/>

²<https://www.tensorflow.org/>

³<https://huggingface.co/>

legalita, komfort a výdělek. Prostředím je v tomto případě silnice, chodci nebo zákazníci. Aktuátorem v této situaci rozumíme volant, plyn, brzdu, spojku ap. A nakonec senzory mohou být například kamery, sonar, tachometr, senzory motoru ap.

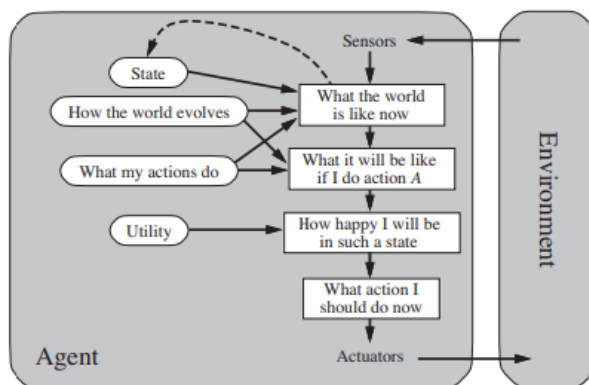
Multiagentním systémem rozumíme skupinu volně propojených agentů, kteří navzájem spolupracují v zájmu dosažení konkrétního cíle. Tato schopnost činí multiagentní systémy ideální pro řešení komplexnějších a hlavně dynamických problémů.

V kontextu generování kódu může být agent navržen různými způsoby:

- **Generátor kódu** – na základě požadavků generuje kód
- **Validátor** – kontroluje syntaktickou a sémantickou správnost vygenerovaného kódu
- **Optimizér** – navrhuje úpravy a optimalizace kódu pro lepší efektivitu nebo čitelnost
- **Koordinátor** – řídí spolupráci ostatních agentů v systému

Většina dnes navrhovaných a používaných modelů (GPT, BERT, LLaMA) jsou tzv. *monolitické*. To znamená, že mají jednu promptovou bránu a jeden výstup. Nejsou tedy rozděleny na spolupracující komponenty a veškerá logika se děje „uvnitř“. Jejich účelem je také přijít s výsledkem co nejrychleji a příliš neztrácet čas případnými vedlejšími faktory.

Oproti tomu agentní přístup staví na koncepci více spolupracujících entit (agentů), z nichž každý může být zodpovědný za konkrétní část procesu, například za generování, kontrolu validity, opravu nebo rozhodování. Agentní architektura tak přináší větší modularitu, možnost paralelizace a potenciálně lepší kontrolu nad jednotlivými kroky výpočtu. Tento přístup je méně běžný, ale v kontextu složitějších nebo bezpečnostně citlivých úloh může nabídnout významné výhody.



Obrázek 5.1: Jednoduchá vizualizace agentního systému – Na obrázku vidíme agentní systém, jehož agent je na levé straně obrázku. Ten na základě různých podnětů mění stav systému. Tomuto konkrétnímu modelu se také říká *Utility based agent*.⁴

5.1.1 Struktura agentů

Úlohou umělé inteligence je navrhnout tzv. *agentní program* (*agent program*), který implementuje funkci agenta. Očekáváme, že program poběží na nějaké výpočetní jednotce se senzory a aktuátory, což se nazývá *architektura*:

⁴Převzato z [29]. O utility based agentech více pojednávám v následující podkapitole 5.1.1.

$$\text{agent} = \text{architektura} + \text{program} \quad (5.1)$$

Každý agent podle konkrétního programu kombinuje konkrétní komponenty určitým způsobem. Agenty podle agentního programu dělíme na několik kategorií:

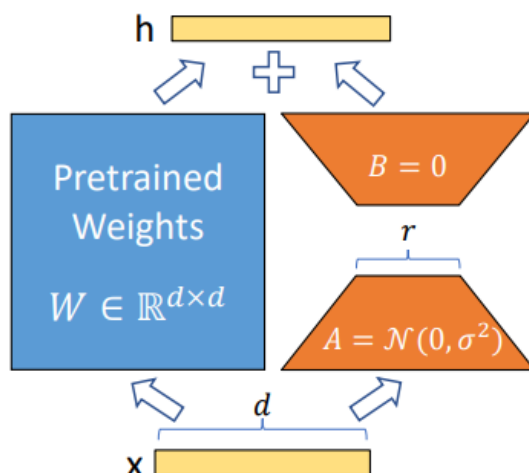
- **Simple reflex agents** – Jednoduší reflexní agenti. Jedná se o nejjednodušší typ. Tito agenti vybírají akce na základě aktuálního vjemu (vstupu), přičemž ignorují zbytek historie vjemů.
- **Model-based reflex agents** – Pro zvládnání *částečné pozorovatelnosti* si agent udržuje *vnitřní stav*, který vychází z historie vjemů a pomáhá mu odhadovat části světa, které právě nevidí. Tento stav může obsahovat například dříve odmítnuté vygenerované části kódu nebo známá bezpečnostní rizika. Aby ho agent dokázal aktualizovat, musí vědět, jak se svět mění sám o sobě a jak jeho vlastní akce svět ovlivňují. Agenti, kteří tuto znalost využívají, se nazývají agenti založení na modelu.
- **Goal-based agents** – Agenti založení na cíli. Znalost aktuálního stavu prostředí nemusí stačit k rozhodnutí, jak se má agent zachovat. Pokud bude přáním uživatele například: „Vytvoř mi funkci pro seřazení seznamu.“, potřebuje agent pro správné rozhodnutí znát cíl uživatele. Může se lišit důraz uživatele na efektivitu, na kompatibilitu s určitým prostředím nebo na jednoduchost funkce. Tento cíl ovlivňuje výsledné řešení.
- **Utility-based agents** – Ani samotné cíle nemusí stačit, aby agent vykazoval kvalitní chování. Cíl pouze rozlišuje mezi tím, jestli byl úkol splněn, nebo ne. Pro detailnější hodnocení kvality výstupu se zavádí pojem *utility* (*užitečnost*), která pomáhá odlišit například dobře optimalizovaný, srozumitelný nebo bezpečný kód od méně kvalitních řešení. Agent pak používá tzv. *užitkovou funkci*, která vyjadřuje, jak si jednotlivé varianty řešení stojí podle stanovených kritérií (např. správnost, čitelnost, časová složitost). Pokud je tato funkce v souladu s tím, co od něj očekává uživatel nebo externí metrika hodnocení, bude agent považován za racionálního.

5.2 Quantized Low-Rank Adaptation

Quantized Low-Rank Adaptation (QLoRa) je rozšířením metody *Low-Rank Adaptation* (LoRa). Obě metody jsou navrženy pro efektivní a paměťově méně náročné doladění (fine-tuning) velkých jazykových modelů. Tradičně bylo nutno aktualizovat obrovské množství parametrů, což je náročné paměťově i výpočetně.

Práce zavádějící metodu LoRa [14] uvádí, že LoRa je metoda, která zmrazuje váhy předtrénovaného modelu a přidává trénovatelné nízkorozměrové matice do každé vrstvy architektury Transformeru. Tím se výrazně snižuje počet parametrů, které je potřeba trénovat pro následné úkoly. Ve srovnání s jemným doladěním modelu GPT-3⁵ pomocí Adam optimizéru, LoRA snižuje počet trénovatelných parametrů 10 000krát a snižuje požadavky na GPU paměť třikrát. I přesto, že má LoRA méně trénovatelných parametrů, dosahuje podobné, nebo dokonce lepší kvality modelu na úlohách jako RoBERTa, DeBERTa, GPT-2 a GPT-3, a to při vyšším výkonu trénování a bez přidání latence během inference (na rozdíl od adaptérů).

⁵Ten má přibližně 175 miliard parametrů.[5].



Obrázek 5.2: Vizualizace reparametrizace podle metody LoRa – metoda trénuje pouze A a B .⁶

LoRa nabízí další velmi zásadní výhodu. Předtrénovaný model lze sdílet a použít pro sestavení mnoha malých LoRA modulů pro různé úlohy. Sdílený model můžeme „zmrazit“ (tedy netrénovat) a efektivně přepínat mezi úlohami pouhou výměnou matic A a B (viz 5.2), čímž se výrazně snižuje paměťová náročnost i režie při přepínání úloh. Tato výhoda čini Lora ideálním nástrojem pro vývoj multiagentních systémů s více Transformersy.

Metoda se ještě zefektivní, pokud ji rozšíříme o tzv. *kvantizaci*, tedy převod některých modelových vah do méně přesného formátu (například na 4bitovou reprezentaci), čímž dále snižujeme paměťové nároky. Tomuto rozšíření pak říkáme *Quantized Low-Rank Adaptation* (QLoRA)[7]. Podle zaváděcí práce lze doladit kvantizovaný model s 4bitovou přesností bez jakékoliv ztráty výkonu.

Kombinací LoRa a kvantizací lze ladit i obrovské modely na běžném hardwaru. Práce [7] uvádí příklad, kdy ladění modelu LLaMa, která má 65 miliard parametrů, vyžaduje 780 GB paměti GPU. QLoRa snižuje paměťovou náročnost na 48 GB, aniž by došlo k degradaci výpočetního výkonu nebo přesnosti predikce ve srovnání s plně doladěným 16bitovým modelem.

5.3 Použité datové sady

Datové sady jsou klíčové pro trénování a hodnocení modelů. Poskytují potřebné informace pro učení, testování a validaci. V následujících podkapitolách představím datové sady, které využívám pro tento projekt.

5.3.1 Generování kódu

Hlavní experimenty byly prováděny s 2 datovými sadami. Jako datovou sadu pro generování kódu jsem využil existující *CodeXGLUE text-to-code* [20]. Tato datová sada pochází z korpusu *CodeSearchNet*[15], který obsahuje velkou datovou sadu dokumentovaných funkcí v jazyce python o velikosti přes půl milionu záznamů. Korpus získává funkce v jazyce python z

⁶Převzato z [14].

veřejně dostupných open-source GitHub repozitářů ⁷. Přestože se jedná o kvalitní datovou sadu, pro účely projektu je příliš velká (nabízí 251 820, 9 604 a 19 210 vzorků pro trénink, validaci a testování.) a dokumentuje i více než stořádkové metody, což je nežádoucí.

Pro ladění generátoru byla nakonec využita datová sada *iamtarunpython_code_instructions_18k_alpaca*⁸. Jedná se o veřejně dostupnou datovou sadu s dokumentovanými metodami v jazyce python. Obsahuje přibližně 18 000 záznamů, z čehož 90 % bylo využito jako tréninková sada a zbylých 10 % jako evaluační sada.

5.3.2 Korekce chyb

Pro účely tvorby datové sady pro opravy chyb byl vytvořena vlastní datová sada získaná z veřejně přístupných repozitářů na GitHub.

S využitím knihovny `GitPython` dojde v první řadě ke naklonování vybraného repozitáře, načte algoritmus projde jeho historii committů a identifikuje změny, které by mohly souviset s opravami kódu. To dělá tak, že vyhledává klíčová slova jako „fix“, „bug“, „error“ nebo „issue“ v popisu commitu.

Repozitáře jsou získány v první řadě na základě jazyka. Jsou pak seřazeny podle udělených hvězd sestupně. Celkově bylo nalezeno tímto způsobem 17 196 943 repozitářů, z nich bylo použito 100.

Když je nalezena změna, následně extrahuje rozdíly ve zdrojových souborech s příponou *.py* a pomocí regulárních výrazů identifikuje ty metody, které byly mezi jednotlivými verzemi upraveny.

Dalším omezením je počet upravených řádků. Množství řádků, které byly v commitu upraveny, nesmí přesáhnout sedm. To slouží k omezení šumu a předejde se také učení modelu příliš velkými změnami, které by mohly měnit i celkovou logiku kódu, což není žádoucí. Je také velmi pravděpodobné, že tak model také nebude laděn pomocí dat, které přidávají úplně novou funkcionalitu.

Stáří commitů také nesmí přesáhnout více než 3 roky, aby se předešlo využíváním oprav z příliš starých a hlavně neaktivních projektů. Algoritmus také prochází pouze větve s názvy „main“, „master“ a „stable“.

Vzniklé dvojice chybného a opraveného kódu jsou uloženy ve formátu JSON Lines pro další využití při trénování odpovídajícího modelu.

5.4 Evaluační metody

Ve výčtu nástrojů nesmí chybět ani nástroj závěrečný, tedy způsob, jak finální produkt otestovat i v návaznosti na dnes využívané modely.

5.4.1 Bleu

BLEU (*Bilingual evaluation understudy*)[26] vychází ze základního předpokladu, že čím blíže je strojový překlad profesionálnímu lidskému překladu, tím kvalitnější překlad je. Byl původně vyvinut pro porovnávání překladačů, ale dobře poslouží i v tomto projektu pro evaluaci vygenerovaného kódu na konci agentního systému.

Základním pilířem této metriky je *míra přesnosti* (*precision*). Metrika spočítá, kolik n -gramů z generovaného výstupu (*kandidáta*) se shoduje s referenčním výstupem. Pro $n = 1$,

⁷<https://github.com/>

⁸Dostupné z https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca

tedy *unigramy*, to znamená, kolik jednotlivých tokenů z generovaného kódu se nachází v referenčním kódu.

Dále BLEU používá tzv. *clipped counts*. To zabraňuje nadhodnocení opakovaných slov v kandidátovi, pokud se v referenci vyskytují méněkrát.

Běžně se BLEU vyhodnocuje pro $n = \{1, 2, 3, 4\}$. Následně se bere geometrický průměr získaných přesností.

BLEU navíc využívá penalizaci *blevity penalty* (BP) pro případy, kdy je kandidát kratší než reference. To pomáhá zabránit modelu, aby generoval krátké výstupy. BP se počítá následovně:

$$\text{BP} = \begin{cases} 1 & \text{pro } c > r \\ e^{(1-r/c)} & \text{pro } c \leq r \end{cases} \quad (5.2)$$

kde c je délka kandidáta a r je délka reference. Celý výpočet poté vypadá následovně:

$$\text{Bleu} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right) \quad (5.3)$$

kde p_n počet n -gramů kandidáta, které se shodují s referencí a w_n je váha, která je přiřazena každému n -gramu. Ta je typicky počítána jako $w_n = \frac{1}{N}$.

Metrika BLEU se pohybuje v rozmezí od 0 do 1. Jen málokterý překlad dosáhne skóre 1, pokud není zcela totožný s referenčním překladem. Z tohoto důvodu ani lidský překladatel nemusí nutně dosáhnout skóre 1. Je důležité poznamenat, že čím více referenčních překladů na jednu větu existuje, tím vyšší může být dosažené skóre. Proto je nutné být opatrný při porovnávání výsledků hodnocení, které využívají různý počet referencí. Například na testovacím korpusu obsahujícím přibližně 500 vět (40 zpráv z běžného zpravodajství) dosáhl lidský překladatel skóre 0,3468 proti čtyřem referencím, ale pouze 0,2571 proti dvěma referencím.

Kapitola 6

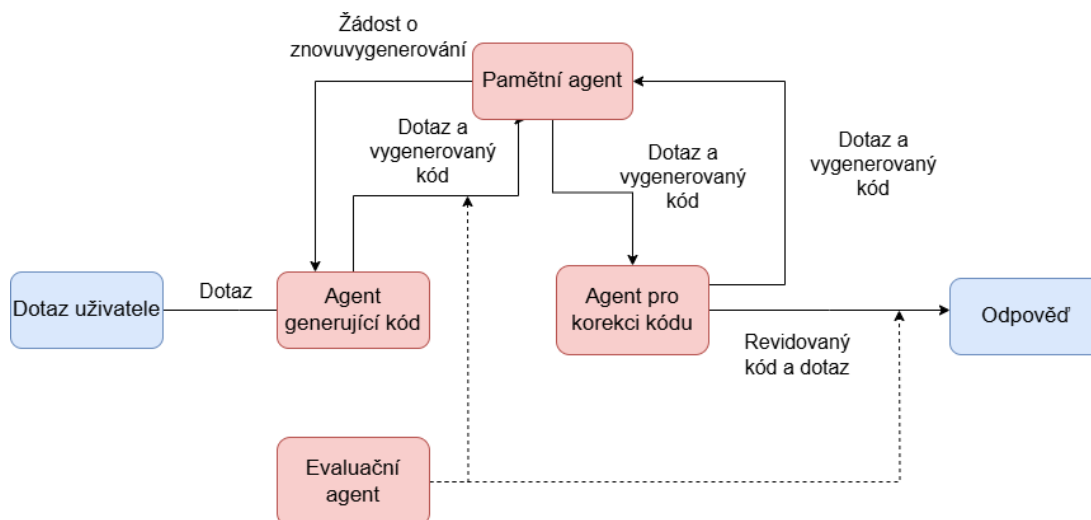
Návrh

Nástrojů, které jsou postaveny na umělé inteligenci, je nyní k dispozici velké množství. Většina dnes přístupným nástrojů je založeno na jednom vstupu uživatele a jednom výstupu. Jednou z možností, které však příliš prozkoumány ještě nejsou, je přistupovat ke generování kódu s využitím agentních systémů, kde hned několik spolupracujících agentů vykonává specializované úkoly, čímž se může zvýšit kvalita a efektivita kódu za cenu vyšších nároků na výpočetní techniku.

Cílem tohoto přístupu je vytvořit dynamický a adaptivní systém, který generuje kvalitní kód, učí se z uživatelské zpětné vazby a vyvíjí se na základě jak úspěchů, tak i neúspěchů. Tento metodologický přístup pomáhá zefektivnit pracovní procesy a umožňuje uživatelům vytvářet spolehlivý a optimalizovaný kód s minimálním manuálním zásahem.

6.1 Architektura systému

Systém bude multiagentním systémem, jehož vstupem bude uživatelský dotaz v přirozeném jazyce v angličtině a výstupem bude kód v jazyce python. Cílem systému kromě prvotní generace kódu bude i využít následně odlišný Transformer pro opravu případných chyb. V systému se také bude nacházet paměťový agent, který umožní uživateli personalizovat vygenerované kódy pro přívětivější uživatelský zážitek a lehčí práci v budoucnu. Systém doplní evaluační agent, který poslouží zejména pro vyhodnocení kvality výstupního kódu.



Obrázek 6.1: Blokový diagram aplikace CodeAgent. Červeně jsou označeny součástky multiagentního systému. Jednotlivé komponenty jsou blíže popsány v dalších kapitolách.

6.1.1 Kód generující agent

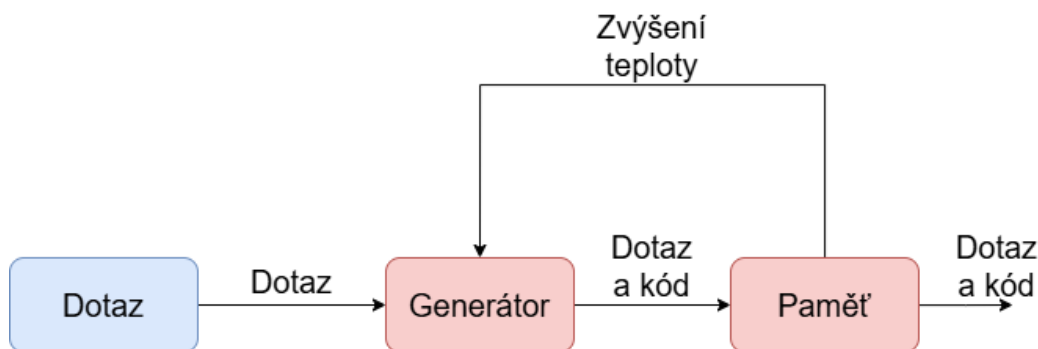
Prvním úkolem aplikace CodeAgent a zároveň vstupem do celkového multiagentního systému je vygenerovat kód na základě dotazu uživatele.

První experimenty byly prováděny pomocí předtrénovaného modelu. CodeBERT[10] trénovaný na datové sadě text-to code CodeXGLUE[20] pomocí metody QLoRa s optimalizací Adam. Byl původně zvolen, protože byl speciálně trénován na dvojicích kód + přirozený jazyk (např. Python + anglický popis). Přes tuto výhodu ale není vhodný pro generování kódu, protože postrádá dekodér. Je spíše vhodný pro pozorování a sumarizaci kódu či případné vyhledávání chyb. Generování kódu pomocí tohoto modelu i přes ladění rozsáhlou datovou sadu nepřineslo kýžený efekt.

Jako model pro generování kódu byl následně zvolen CodeGen[25], konkrétně verze codegen-350m-mono, protože byl předtrénován na rozsáhlých datových sadách v jazyce python a současně se jedná o model s relativně malým počtem parametrů (asi 350 milionů).

Uživatel bude aplikaci požádat, aby zadal úlohu, na základě které tento agentní systém bude generovat kód. Výstup tohoto konkrétního agentního systému ale nebude výstup celku. Na základě vstupních argumentů bude systém rozlišovat tři stavy:

1. Multiagentnímu systému není přidělen pamětní agent. Výsledný kód bude diskutován pouze s korekcí kódu.
2. Multiagentní systém obsahuje pamětního agenta, který nerozliší žádnou podobnost s kódy v paměti. Výsledný kód je dále zpracován korekcí.
3. Multiagentní systém obsahuje pamětního agenta, který rozliší dříve odmítnutý nebo opravený kód. V takovém případě žádá generátor o přegenerování s úpravami vstupních argumentů, zejména teploty.



Obrázek 6.2: Blokový diagram logiky generátoru a paměťového agenta s využitím modelu CodeGen.

6.1.2 Pamětní agent

Paměťový agent slouží jako trvalý uchovatel zpětné vazby v multiagentním systému. Ukládá odmítnuté nebo opravené kódy a jejich kontext, čímž pomáhá předcházet opakování stejných chyb při budoucím generování. Tento agent neprovádí aktivní opravy, ale poskytuje cenné informace ostatním částem systému. Například upozorňuje generátor, pokud navržený kód připomíná buď dříve odmítnuté řešení anebo korektorem opravené řešení.

Jedná se o volitelnou součást CodeAgent, která posiluje agenta opravující kód a personalizuje prvotní generování kódu. Odmítnuté a opravené kódy jsou uloženy na lokálním disku ve formě JSON. Záznamy obsahují vstup klienta, vygenerovaný kód, typ záznamu¹ a časová známka. Ukázka záznamu viz na obrázku 6.3.

S pomocí knihovny `difflib` je vygenerovaný kód porovnán s pamětí a případná shoda je signál pro generátor k opětovnému vygenerování. Nutno dodat, že shoda musí být mezi kódy i dotazy.

```

{
  "prompt": "Write a Python function to check if a number is prime.",
  "generated_code": "def is_prime(n):\n    for i in range(2, n):\n        if n % i == 0:\n            return False\n        return True",
  "status": "declined",
  "timestamp": "2025-05-11T13:00:00"
},
{
  "prompt": "Write a Python function to check if a number is prime.",
  "generated_code": "def is_prime(n):\n    if n <= 1:\n        return False\n    for i in range(2, int(n ** 0.5) + 1):\n        if n % i == 0:\n            return False\n        return True",
  "status": "corrected",
  "timestamp": "2025-05-11T13:05:00"
}

```

Obrázek 6.3: Ukázka záznamu pamětního agenta – Obsahuje dva záznamy; jeden odmítnutý kód a jeden opravený kód.

V případě žádné shody s pamětním agentem kód pokračuje dále do systému. V případě shody s minulými generacemi požádá systém generujícího agenta a přeregenerování kódu se zvýšenou teplotou.

6.1.3 Agent pro korekci kódu

Agent pro korekci kódu využívá model CodeBERT. Tento agent slouží k detekci syntaktických i sémantických chyb v automaticky generovaném kódu. Pomocí jemného doladění a

¹Kód mohl být odmítnut nebo opraven.

datech obsahujících chybný a opravený kód 5.3.2 se model naučí identifikovat vzory typických chyb a navrhnout odpovídající opravy. V praxi agent přijímá výstpní kód generátoru, analyzuje jej ve formě tokenizovaného vstupu a následně generuje upravenou verzi, která odpovídá správné implementaci dané úlohy. Díky předtrénování na rozsáhlých programovacích korpusech dokáže CodeBERT rozpoznat běžné konstrukce, identifikovat překlepy, nesprávné proměnné či chybějící bloky a vylepšit tak celkovou kvalitu výstupu systému.

Pro tento modul byla využita kvantizace pomocí QLoRa pro snížení náročnosti a ceny trénování a inference.

6.1.4 Agent pro evaluaci kódu

Pro účely experimentů a ohodnocení kvality výsledků jsem se rozhodl vyhodnocovat výsledky nejenom jako výstup celého CodeAgent, ale i po úvodním vygenerování kódu modelem CodeGen.

CodeAgent bude testován oproti dnes dostupným nástrojům, abych mohl vyhodnotit jeho výkon a použitelnost. Jako základní referenci budu využívat GPT-4o od OpenAI.

Očekávaným přínosem CodeAgent, který však lze otestovat pouze uživatelem, je personalizace na základě odmítnutých kódů. V kapitole 8 je však i na tento případ myšleno.

Kapitola 7

Implementační detaily

Implementace CodeAgent lze rozdělit na 2 části. První částí jsou přípravné, trénovací a vyhodnocující soubory ve formě Python a shell skriptů. Druhou částí je samotná aplikace, kde lze výsledný produkt po vyladění modelů vyzkoušet.

7.1 Trénink a vyhodnocování

V projektu CodeAgent probíhá trénink pomocí skriptů `train_*.py`. Ten se dá spustit přímo, ale pro jeho spuštění slouží i skripty `train_*.sh`, které obsahují potřebné proměnné pro správné ladění modelů. Argumenty jsou zpracovány pomocí modulu `argument_parser`. Na základě modelu je načten tokenizér a předtrénovaný model pomocí knihovny `transformers`.

Datové sady, které slouží ke správnému ladění modelů, a jejich získávání je popsáno v předešlé sekci 5.3.

7.1.1 Načtení dat

Skripty `train_*.py` jsou vstupním bodem a zároveň základním pilířem celého tréninkového procesu. Načítání argumentu probíhá pomocí modulu `argument_parser.py`. Ten přijímá všechny pro trénink potřebné argumenty jako název předtrénovaného modelu, maximální délka sekvence tokenů, zdroj datové sady, počet trénovacích epoch, míra učení a jiné.

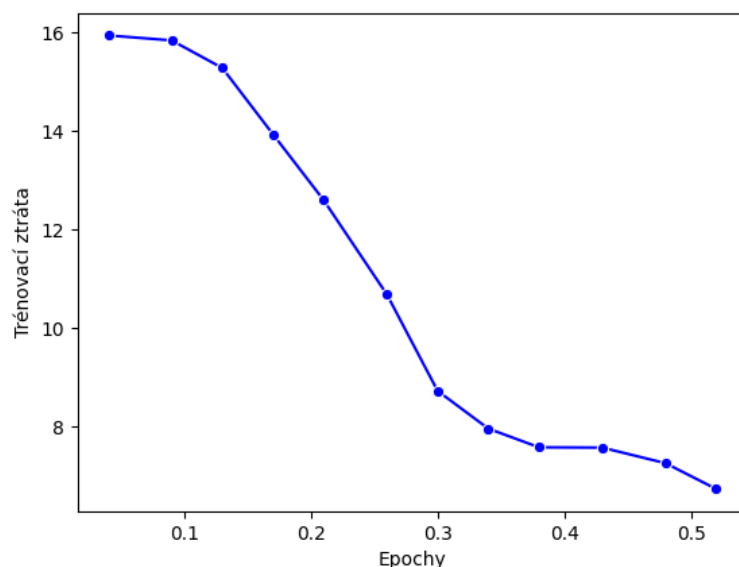
Na model jsou aplikovány optimalizační metody jako např. LoRa pro snížení nutného počtu parametrů, které je třeba trénovat.

Ze získaných datových sad jsou extrahována potřebná data, pro generátor se jedná o vstupní instrukce uživatele a vygenerovaný kód a pro korektor se jedná o vstupní kód s chybou a výstupní opravený kód. Zde je použit modul `dataset_loader.py`. Jeho funkce slouží k transformaci dat do podoby vhodné pro trénování modelu strojového učení. Každý vstup je převeden na číselnou reprezentaci pomocí tokenizace, přičemž je zajištěno zarovnání délky a vytvoření masek pro pozornost. Tímto způsobem je připraven jednotný formát vstupů a výstupů, který může být efektivně zpracován během trénovacího procesu. Výsledkem je kolekce strukturovaných prvků, které zachycují podstatné informace potřebné pro učení modelu.

^{0*} je náhrada za odpovídající model.

7.1.2 Trénink

V této části je vytvořena instance trénovací konfigurace pomocí objektu *TrainingArguments*, který specifikuje klíčové parametry pro řízení procesu učení. Obsahuje nastavení velikosti dávky pro trénování i vyhodnocování, počet epoch, krok akumulace gradientu, rychlost učení, i další optimalizační strategie, jako je warm-up fáze, regularizace pomocí váhového útlumu a použití výpočtů s plovoucí desetinnou čárkou v poloviční přesnosti (fp16) pro vyšší efektivitu. Součástí konfigurace jsou také intervaly pro logování, ukládání modelu a vyhodnocování jeho výkonu, což zajišťuje pravidelný dohled nad průběhem učení a bezpečné uchování průběžně trénovaných váh. Cílem tohoto nastavení je zajistit stabilní a efektivní tréninkový proces přizpůsobený možnostem výpočetního prostředí i charakteru použitého modelu.



Obrázek 7.1: Graf vztahu ztráty k postupu epochami podle parametrů v sekci 7.1.2. Model byl laděn na jednom GPU GeForce RTX 4080. Proces trval přibližně 8 h a 34 m. Graf znázorňuje pouze první polovinu první epochy.

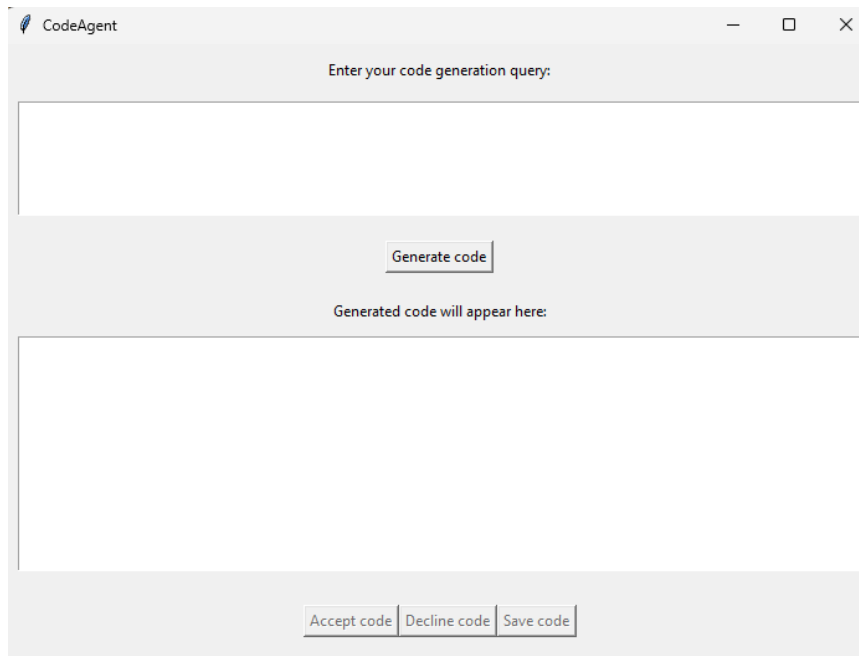
Pro trénink jsem zvolil následující hodnoty parametrů.

per_device_train_batch_size a *per_device_eval_batch_size* jsem nastavil na hodnoty 8 a 16, což je běžná volba pro středně velké modely při použití GPU.

gradient_accumulation_steps je nastaveno na 4, což umožňuje efektivně zvětšit velikost dávky bez přetížení paměti. Epochy jsem zvolil tři a *warmup_steps* jsem nastavil na hodnotu 1000, aby se míra učení během trénování hladce zvyšovala. Pro regulaci přeučení je nastaven *weight_decay* na 0.002. Ukládání checkpointů probíhá každých 1000 kroků.

7.2 Aplikace

Pro účely vyzkoušení bylo vytvořeno jednoduché uživatelské rozhraní, které umožňuje uživateli zadat dotaz do textového pole, aby si mohl vygenerovat kód pomocí CodeAgent.



Obrázek 7.2: Ukázka aplikace CodeAgent

Start aplikace je veden TOML configuračním souborem, který určuje, kde se nachází vyladěné modely a kde se nachází paměť CodeAgent. Modely jsou nezbytné pro správné fungování aplikace a jsou uloženy lokálně.

Modely (a paměť systému) jsou načteny současně s aplikací do operační paměti, aby byly kdykoliv k dispozici. Vzhled aplikace vypadá po zapnutí jako na obrázku 7.2. Pro správné fungování je však nutné první modely natrénovat jak jest popsáno v 7.1.

Po vygenerování kódu má uživatel možnost generovaný kód přijmout nebo odmítnout. Odmítnutím informuje CodeAgent, že tento kód není žádoucím nebo vhodným výsledkem. Tuto informaci systém v dalších generacích zohlední i po znovuzapnutí aplikace. Aplikace umožňuje i vygenerovaný kód uložit.

7.2.1 Logika aplikace

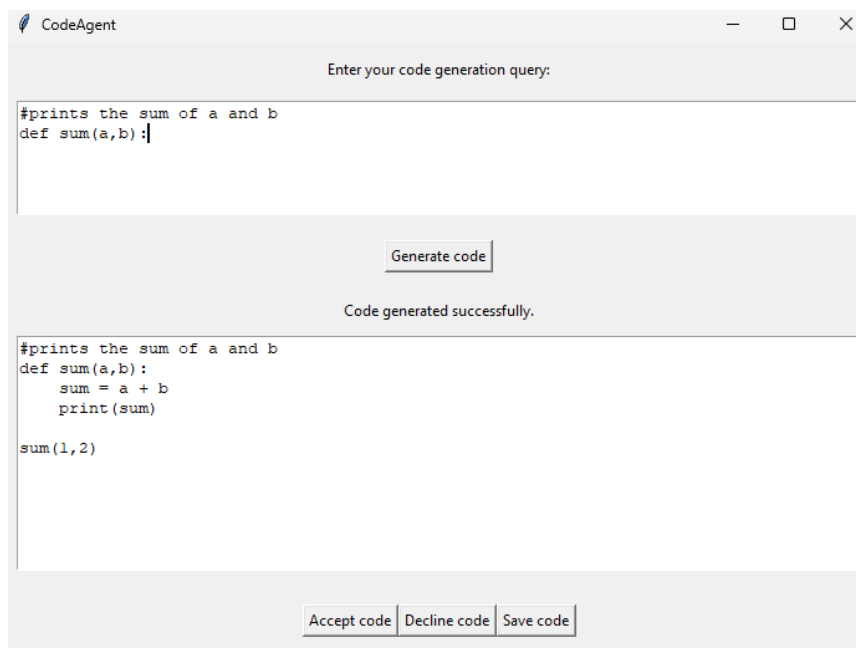
Hlavní nástroje aplikace se nachází v modulu `app_utils.py`. Podle stavu paměti se dělí logika na dva hlavní proudy:

- Paměť není zpřístupněna. Po vložení instrukce dojde ke generaci kódu a tisku výstupu do uživatelského rozhraní aplikace.
- Paměť je zpřístupněna. Kód se generuje do té chvíle, dokud pamětní agent neurčí kód jako unikátní oproti všem ostatním.

7.2.2 Paměť

Jedním s cílů projektu bylo vytvořit personalizované generování kódu, které se mění v závislosti na odmítnutých a opravených kódech. Pro tento účel slouží modul `memory.py` a úložiště `memory.json`.

Úkolem modulu je uchovávat a poskytovat paměť systému. Samotná logika vyhodnocování podobnosti se již nachází v modulu `app_utils.py` 7.2.1. Hraniční hodnota se definuje v konfiguračním souboru `config.toml`.



Obrázek 7.3: Ukázka generování kódu

Kapitola 8

Vyhodnocení výsledků

Na závěr musí být aplikace CodeAgent otestovaná i v návaznosti na běžně používané nástroje, aby mohl být vyhodnocen její přínos.

8.1 Vyhodnocení generování kódu

Po lidském vyhodnocení je zřejmé, že kódy GPT-4o mají mnohem lepší výkon s výjimkou jednoduchých algoritmů. V této oblasti CodeAgent taktéž generuje smysluplné a přehledné kódy. Pro složitější zadání však již CodeAgent může konkurovat jen zřídka. To samé platí i pro strojové vyhodnocení metrikou BLEU. V krátkých kódech dosahoval CodeAgent přibližně 75 % hodnocení oproti konkurenčním 95 %. Pro složitá zadání však hodnocení klesá až k 15 %.

8.2 Vyhodnocení systému

Přestože byl v rámci multiagentního systému implementován specializovaný agent pro opravu kódu, jeho výstupy byly často nerelevantní vzhledem k původnímu kódu. Ve více případech agent navrhl syntakticky i sémanticky nesouvisející opravy i v případě zcela správného kódu, což naznačuje, že měl potíže správně identifikovat záměr původního úryvku nebo povahu chyby. To poukazuje na potřebu buď lepší strategie doladění modelu na datové sadě, nebo integrace mechanismů pro lepší kontextové porozumění, například klasifikace typu chyby nebo užší propojení s agentem pro generování kódu.

8.3 Vyhodnocení personalizace

V této části bylo provedeno několik ručních testů, zda aplikace splňuje v návrhu požadovanou funkčnost personalizace generování.

První test byl proveden na výpočtu faktoriálu.

Kapitola 9

Závěr

Práce se zabývala návrhem a implementací multiagentního systému CodeAgent, jehož cílem bylo pomocí agentního přístupu ke generování kódu personalizovat používání generátorů zdrojového kódu a posílit prevenci menších vad. Bylo taktéž vytvořeno uživatelské rozhraní aplikace.

Práce pokrývá základní teoretický základ související s generováním kódu. Podrobně rozebírá neuronové sítě, věnuje se zpracování přirozeného jazyka, velkým jazykovým modelům i architekturám Transformer.

Aplikace CodeAgent představena v této práci má za úkol generovat kód na základě vstupních komentářů nebo instrukcí uživatele tak, že reflektuje dříve vygenerované nebo nežádoucí kódy. Pro účel posílení prevence vad byl vytvořen skript pro tvorbu datové sady. Tenhle agent bohužel nesplňuje očekávané chování a produkuje irelevantní opravy.

Model byl porovnán s GPT-4o, což je model dnes využíván v GitHub copilot. Podle metriky BLEU dosahoval CodeAgent podobného skóre, avšak ruční kontrola výstupu ukázala, že výsledný kód zejména u složitějších popisů je často nesprávný. U jednodušších funkcí vykazuje CodeAgent stabilně kvalitní výkon i při využití paměti a personalizace.

Cílem práce bylo vytvořit multiagentní systém integrující moderní přístupy v oblasti umělé inteligence do kompaktního uživatelského rozhraní. Celý přístup otevírá možnosti dalšího vývoje směrem k adaptivnímu generování kódu, které se dokáže přizpůsobit požadavkům projektu, preferencím vývojáře nebo stylu kódování. Budoucí vývoj by se měl zaměřit na možné přídatné moduly systému, jako například filtr škodlivého kódu nebo hodnotící modely. Posílením personalizace, či integrací získaných kódů do datové sady by také došlo ke zlepšení celkového zážitku uživatele.

Literatura

- [1] AFAQ, S. a RAO, S. Significance Of Epochs On Training A Neural Network. *International Journal of Scientific & Technology Research*, 2020, sv. 9, s. 485–488. Dostupné z: <https://api.semanticscholar.org/CorpusID:225647672>.
- [2] BAHDANAU, D.; CHO, K. a BENGIO, Y. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2016. Dostupné z: <https://arxiv.org/abs/1409.0473>.
- [3] BIRD, S.; KLEIN, E. a LOPER, E. *Natural Language Processing with Python*. O'Reilly, 2009. Dostupné z: <https://www.nltk.org/book/>.
- [4] BRITZ, D.; GOLDIE, A.; LUONG, M.-T. a LE, Q. *Massive Exploration of Neural Machine Translation Architectures*. 2017. Dostupné z: <https://arxiv.org/abs/1703.03906>.
- [5] BROWN, T. B.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J. et al. *Language Models are Few-Shot Learners*. 2020. Dostupné z: <https://arxiv.org/abs/2005.14165>.
- [6] CHO, K.; MERRIENBOER, B. van; GULCEHRE, C.; BAHDANAU, D.; BOUGARES, F. et al. *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. 2014. Dostupné z: <https://arxiv.org/abs/1406.1078>.
- [7] DETTMERS, T.; PAGNONI, A.; HOLTZMAN, A. a ZETTLEMOYER, L. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. Dostupné z: <https://arxiv.org/abs/2305.14314>.
- [8] DEVLIN, J.; CHANG, M.-W.; LEE, K. a TOUTANOVA, K. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Dostupné z: <https://arxiv.org/abs/1810.04805>.
- [9] DUBEY, S. R.; SINGH, S. K. a CHAUDHURI, B. B. A Comprehensive Survey and Performance Analysis of Activation Functions in Deep Learning. *CoRR*, 2021, abs/2109.14545. Dostupné z: <https://arxiv.org/abs/2109.14545>.
- [10] FENG, Z.; GUO, D.; TANG, D.; DUAN, N.; FENG, X. et al. *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*. 2020. Dostupné z: <https://arxiv.org/abs/2002.08155>.
- [11] GOODFELLOW, I.; BENGIO, Y. a COURVILLE, A. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [12] GRAVES, A. Generating Sequences With Recurrent Neural Networks. *ArXiv*, 2013, abs/1308.0850. Dostupné z: <https://api.semanticscholar.org/CorpusID:1697424>.

- [13] HOCHREITER, S. a SCHMIDHUBER, J. Long Short-Term Memory. *Neural Computation*, Listopad 1997, sv. 9, č. 8, s. 1735–1780. ISSN 0899-7667. Dostupné z: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [14] HU, E. J.; SHEN, Y.; WALLIS, P.; ALLEN ZHU, Z.; LI, Y. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. Dostupné z: <https://arxiv.org/abs/2106.09685>.
- [15] HUSAIN, H.; WU, H.-H.; GAZIT, T.; ALLAMANIS, M. a BROCKSCHMIDT, M. *CodeSearchNet Challenge: Evaluating the State of Semantic Code Search*. 2020. Dostupné z: <https://arxiv.org/abs/1909.09436>.
- [16] JURAFSKY, D. a MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd. 2025. Dostupné z: <https://web.stanford.edu/~jurafsky/slp3/>. Online manuscript released January 12, 2025.
- [17] KINGMA, D. P. a BA, J. *Adam: A Method for Stochastic Optimization*. 2017. Dostupné z: <https://arxiv.org/abs/1412.6980>.
- [18] KUDO, T. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. In: GUREVYCH, I. a MIYAO, Y., ed. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Melbourne, Australia: Association for Computational Linguistics, červenec 2018, s. 66–75. Dostupné z: <https://aclanthology.org/P18-1007/>.
- [19] KUDO, T. a RICHARDSON, J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In: BLANCO, E. a LU, W., ed. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Brussels, Belgium: Association for Computational Linguistics, Listopad 2018, s. 66–71. Dostupné z: <https://aclanthology.org/D18-2012/>.
- [20] LU, S.; GUO, D.; REN, S.; HUANG, J.; SVYATKOVSKIY, A. et al. *CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation*. 2021. Dostupné z: <https://arxiv.org/abs/2102.04664>.
- [21] MARCUS, M. P.; SANTORINI, B. a MARCINKIEWICZ, M. A. Building a Large Annotated Corpus of English: The Penn Treebank. *Computational Linguistics*. Cambridge, MA: MIT Press, 1993, sv. 19, č. 2, s. 313–330. Dostupné z: <https://aclanthology.org/J93-2004/>.
- [22] MCCULLOCH, W. S. a PITTS, W. *A Logical Calculus of the Ideas Immanent In Nervous Activity*. Pergamon Press, 1943.
- [23] MIKOLOV, T.; CHEN, K.; CORRADO, G. a DEAN, J. *Efficient Estimation of Word Representations in Vector Space*. 2013. Dostupné z: <https://arxiv.org/abs/1301.3781>.
- [24] MIKOLOV, T.; YIH, W.-t. a ZWEIG, G. Linguistic Regularities in Continuous Space Word Representations. In: VANDERWENDE, L.; DAUMÉ III, H. a KIRCHHOFF, K.,

- ed. *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Atlanta, Georgia: Association for Computational Linguistics, červen 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>.
- [25] NIJKAMP, E.; PANG, B.; HAYASHI, H.; TU, L.; WANG, H. et al. *CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis*. 2023. Dostupné z: <https://arxiv.org/abs/2203.13474>.
- [26] PAPINENI, K.; ROUKOS, S.; WARD, T. a ZHU, W.-J. Bleu: a Method for Automatic Evaluation of Machine Translation. In: ISABELLE, P.; CHARNIAK, E. a LIN, D., ed. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, červenec 2002, s. 311–318. Dostupné z: <https://aclanthology.org/P02-1040/>.
- [27] RAFFEL, C.; SHAZEER, N.; ROBERTS, A.; LEE, K.; NARANG, S. et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2023. Dostupné z: <https://arxiv.org/abs/1910.10683>.
- [28] RUMELHART, D. E.; HINTON, G. E. a WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, 1986, sv. 323, s. 533–536. Dostupné z: <https://api.semanticscholar.org/CorpusID:205001834>.
- [29] SABHARWAL, A. S. Russell, P. Norvig, Artificial Intelligence: A Modern Approach, Third Edition. *Artif. Intell.*, Duben 2011, sv. 175, s. 935–937.
- [30] SENNRICH, R.; HADDOW, B. a BIRCH, A. Neural Machine Translation of Rare Words with Subword Units. In: ERK, K. a SMITH, N. A., ed. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Berlin, Germany: Association for Computational Linguistics, Srpen 2016, s. 1715–1725. Dostupné z: <https://aclanthology.org/P16-1162/>.
- [31] SRIVASTAVA, N.; HINTON, G.; KRIZHEVSKY, A.; SUTSKEVER, I. a SALAKHUTDINOV, R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 2014, sv. 15, č. 56, s. 1929–1958. Dostupné z: <http://jmlr.org/papers/v15/srivastava14a.html>.
- [32] VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L. et al. *Attention Is All You Need*. 2023. Dostupné z: <https://arxiv.org/abs/1706.03762>.
- [33] WAGNER, R. A. a FISCHER, M. J. The String-to-String Correction Problem. *J. ACM*. New York, NY, USA: Association for Computing Machinery, leden 1974, sv. 21, č. 1, s. 168–173. ISSN 0004-5411. Dostupné z: <https://doi.org/10.1145/321796.321811>.

Příloha A

Obsah paměťového média

```
dataset
  repositories
    repositories.txt
graphs
  Soubory pro tvorbu grafů...
reports
  loss_report.txt
source
  memory
    memory.json
  app_utils.py
  argument_parser.py
  config.toml
  correction_dataset_retriever.py
  dataset_loader.py
  eval.py
  main.py
  memory.py
  prepare.sh
  python_repositories_retriever.py
  README.txt
  requirements.txt
  run.sh
  train_codecorrector.sh
  train_codegenerator.sh
  train_corrector.py
  train_generator.py
documentation
  Soubory LaTeX pro vysázení práce...
thesis.pdf
thesis-print.pdf
```

.